Job Agent — Project Context for Claude Code

What we're building

A personal AI job search agent. It scrapes job boards, scores every job against my profile using Claude, and surfaces only the relevant ones. Built for personal use — just me and my sister. No SaaS, no multi-tenancy, no overengineering.

Core philosophy

- **Get signal fast.** The scraper + AI filter is the highest-value piece. Auto-applying comes later if at all.
- **Incremental.** We're building step by step. Each step is independently runnable before moving to the next.
- **Simple infra.** Runs locally on a laptop. SQLite, not Postgres. No Docker, no Kubernetes, no cloud required.
- **No overengineering.** If it works for two people, it's done.

Agreed build order (7 steps)

We are currently on **Step 1 (complete)**.

1. ✅ Project skeleton + config.yaml + db.py
2. ⬜ Greenhouse scraper (public API, no auth needed)
3. ⬜ HN Who's Hiring scraper (Algolia API)
4. ⬜ Claude scorer (scores unscored DB jobs against profile)
5. ⬜ main.py orchestrator (glues scrape + score + display into one command)
6. ⬜ Streamlit dashboard (local web UI, job table with status column)
7. ⬜ Sister profile support (profiles/ folder, --profile flag)

What exists right now

config.yaml

The user fills this out once. Contains:

- `profile.resume` — plain text resume
- `profile.bio` — 2-3 sentence background summary
- `preferences.titles` — job titles to search for

- `preferences.desired_skills` — skills they want in the role
- `preferences.hard_no_keywords` — auto-skip if job contains these
- `preferences.location` — remote_ok, preferred cities
- `preferences.compensation.min_salary`
- `preferences.yoe` — years of experience
- `scoring.min_display_score` — minimum score (0-100) to show in output

db.py

All SQLite logic lives here. Nothing else touches the DB directly.

Key pieces:

- `Job` dataclass — the canonical data model
- `make_id(company, title, location)` — MD5 hash for dedup
- `init_db()` — creates jobs table, safe to call every run
- `insert_job(job) -> bool` — True if new, False if duplicate
- `save_score(job_id, score, reasons, flags, one_liner)` — writes Claude output
- `get_unscored() -> list[Job]` — jobs without scores yet
- `get_top_jobs(min_score) -> list[Job]` — scored jobs above threshold
- `get_all_jobs() -> list[Job]` — everything, sorted by score
- `count_jobs() -> dict` — total and scored counts

jobs table schema

```sql
```

```
id          TEXT PRIMARY KEY  -- 12-char MD5 hash
title       TEXT
company     TEXT
location    TEXT
url         TEXT
raw_text    TEXT              -- full text sent to Claude for scoring
source      TEXT              -- "greenhouse" | "hackernews"
score       INTEGER           -- 0-100, NULL until scored
reasons     TEXT              -- JSON list e.g. '["Python match", "remote"]'
flags       TEXT              -- JSON list e.g. '["requires 5yr"]'
one_liner   TEXT              -- Claude's one-sentence fit summary
status      TEXT DEFAULT 'new' -- new | applied | skipped
created_at  TIMESTAMP
```

Sources we're scraping

Greenhouse (Step 2)

- Public REST API: `https://boards-api.greenhouse.io/v1/boards/{slug}/jobs?content=true`
- No auth, pure JSON, very stable
- We maintain a list of company slugs in config.yaml
- Covers most YC-backed and startup companies

HN Who's Hiring (Step 3)

- Monthly thread scraped via Algolia search API
- Each top-level comment = one job posting
- Free, no auth, good signal for early-stage companies

We are deliberately NOT scraping LinkedIn or Indeed right now.
LinkedIn has aggressive bot detection and ToS risk.
These two sources cover the target company profile well enough for now.

Scoring approach (Step 4)

Each job's `raw_text` + the user's full profile gets sent to Claude. Claude returns JSON: `{ score, reasons, flags, one_liner }`

Score interpretation:

- 85-100: near-perfect fit

- 70-84: strong fit
- 60-69: decent, worth a look
- below 60: filtered out of display (still stored in DB)

Scoring weights (configurable in config.yaml):

- role_fit: 0.35
- stack_match: 0.30
- growth_signal: 0.15
- comp_signal: 0.10
- yoe_fit: 0.10

Model: claude-sonnet-4-20250514

Key constraints and decisions

- **No Playwright yet.** That's for a potential future auto-apply step, not in scope now.
- **No auto-applying in current scope.** Tonight's goal is search + rank only.
- **SQLite only.** No Postgres, no Redis, no queue.
- **No Slack notifications yet.** Terminal output is fine for now.
- **Each step must run independently.** Don't couple steps before they're both built.
- **Dedup is hash-based.** Same company + title + location = same ID, rejected on insert.
- **raw_text is capped at ~3500 chars** before being sent to Claude to manage token cost.

Target user profile (for scoring context)

Early-career software engineer, ~2 years experience.
Strong in Python backend, AWS infrastructure, LLM/agent integration.
Looking for: AI engineer, backend engineer, or platform roles.
Targeting startups, especially AI-native companies.
Fully remote or SF/NYC preferred.
Min salary ~$130k.

File structure (current and planned)

```
job-agent/
├── config.yaml        ✅ exists
├── db.py              ✅ exists
├── requirements.txt   ✅ exists
├── scraper.py         ⬜ step 2-3
```

```
├── scorer.py          ▢ step 4
├── main.py            ▢ step 5
├── dashboard.py       ▢ step 6
└── jobs.db            (auto-created on first run)
```

What to work on next

Step 2: Greenhouse scraper

Build `scraper.py` with a `scrape_greenhouse(config)` function that:

1. Reads company slugs and filter preferences from config
2. Hits the Greenhouse API for each company
3. Filters by title match and hard_no_keywords before saving
4. Strips HTML from job descriptions
5. Builds a `Job` object and calls `insert_job()`
6. Prints a summary: X companies checked, Y new jobs saved

Running `python scraper.py` should work standalone and print results.